2. Capa de aplicación

Grado en Ingeniería Informática y Tecnologías Virtuales
Grado en Ingeniería del Software

Software de Comunicaciones

Curso 2025-2026

Universidad LOYOLA

Contenido

1- La web y HTTP

2- DNS

3- *Streaming* de video y redes de distribución de contenido

4- Programación de sockets: creando aplicaciones de red

Contenido

1- La web y HTTP

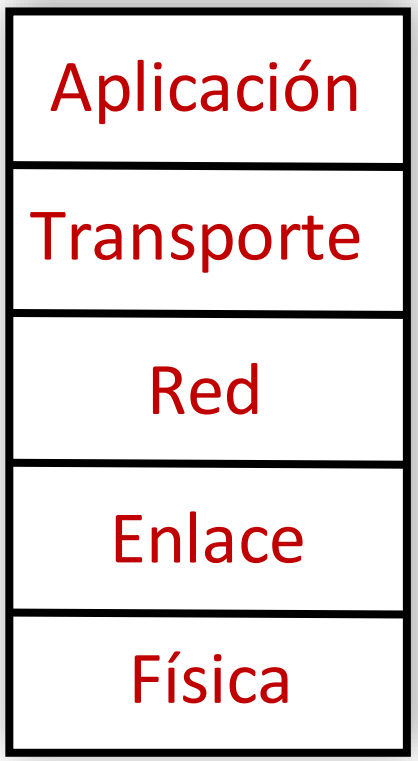
2- DNS

3- *Streaming* de video y redes de distribución de contenido

4- Programación de sockets: creando aplicaciones de red

Arquitectura: modelo de capas

- *Aplicación:* aplicaciones de red
  - HTTP, IMAP, SMTP, DNS, FTP, NTP, DHCP
- *Transporte:* transferencia de datos entre procesos
  - TCP, UDP
- *Red:* enrutado de datagramas desde la fuente al destino
  - IP, ARP, ICMP, IGMP
- *Enlace:* transferencia de datos entre vecinos
  - Ethernet, 802.11 (WiFi), PPP
- *Física:* bits en el medio físico



La web y HTTP

- Hasta principios de 1990, Internet era usado principalmente por investigadores, profesores y estudiantes universitarios para acceder a equipos remotos, transferir archivos entre equipos, enviar y recibir correos electrónicos…
- Una nueva aplicación apareció: La ***World Wide Web***.
  - Primera aplicación que llegó al público general.
  - Desarrollada por Berners-Lee en el CERN (propuso un sistema de gestión de información distribuida basada en Internet).
- **HTTP** (*HyperText Transfer Protocol*). El protocolo de la capa de aplicación de la web. (RFCs 1945, 7230 y 7540).
  - Implementados en dos programas: cliente y servidor.
  - Intercambian mensajes HTTP.

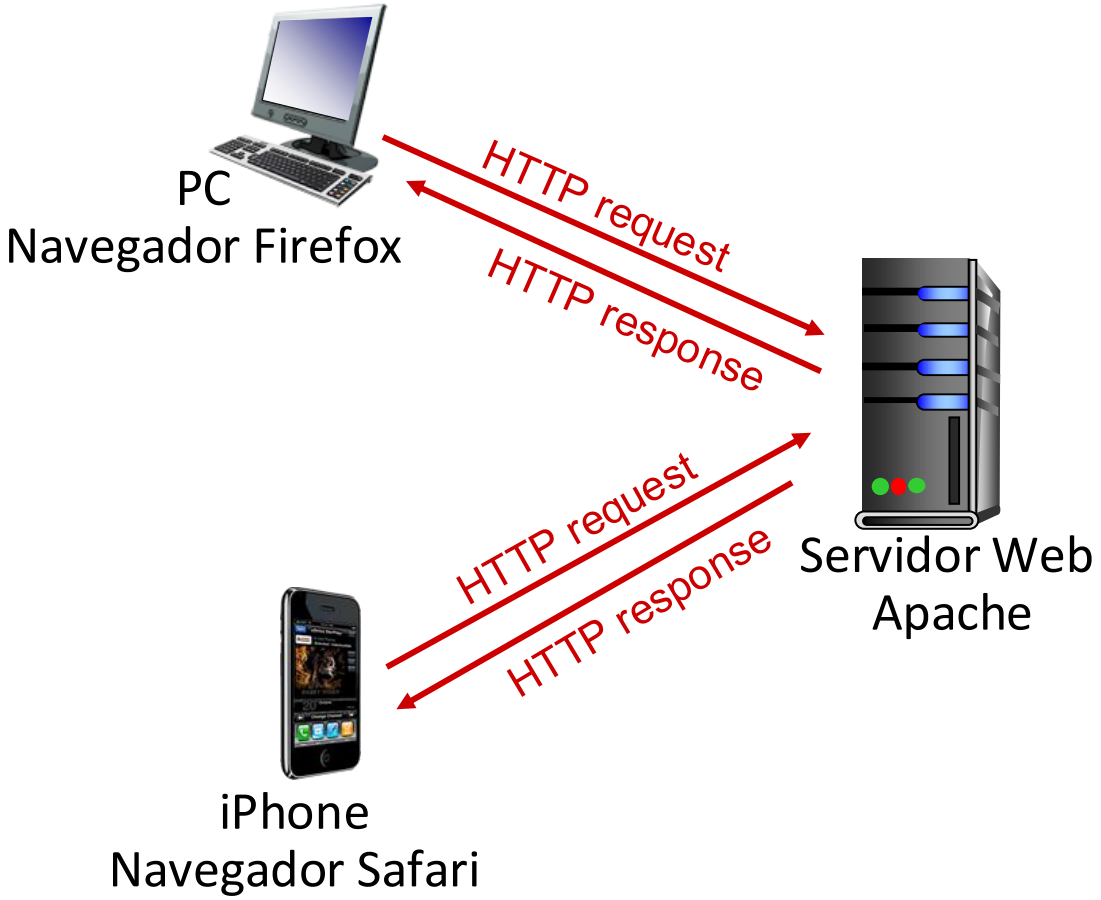
La web y HTTP

- Página web consiste en objetos, cada uno de los cuales pueden ser almacenados en diferentes servidores. Un objeto es simplemente un archivo (archivo HTML, imagen JPEG, archivo JavaScript, archivo de hoja de estilo CSS o un videoclip).
- Mayoría de página web consisten en un archivo **HTML** (*HyperText Markup Language*) base y varios objetos referenciados.
- El archivo HTML referencia los otros objetos en la página con la **URL** (*Uniform Resource Locator*) de los objetos.
- Cada URL tiene dos componentes: el nombre del servidor (*hostname*) que contiene el objeto y la ruta del objeto.

$$\underbrace{\texttt{www.someschool.edu}}_{\textit{hostname}}\underbrace{\texttt{/someDept/pic.gif}}_{\text{ruta}}$$

HTTP

- HTTP: modelo cliente-servidor.
  - **Cliente**: navegador que realiza peticiones (usando el protocolo HTTP) y muestra objetos web.
  - **Servidor**: envía objetos (usando el protocolo HTTP) en respuesta a peticiones.



HTTP

- HTTP usa TCP:
  - Los clientes inician una conexión (crean sockets) TCP al servidor. Puerto 80.
  - Los servidores aceptan una conexión TCP del cliente.
  - Intercambio de mensajes HTTP.
  - Cierre de la conexión TCP.

- Dos tipos de conexiones HTTP:
  - **No persistente**: solo un objeto enviado sobre una conexión TCP.
  - **Persistente**: múltiples objetos pueden ser enviados sobre una conexión TCP.

HTTP

Conexiones no persistentes:

El usuario introduce la URL: www.someschool.edu/someDept/home.index (con texto y referencias a 10 imágenes jpeg)

1- Cliente HTTP inicia conexión al servidor HTTP (proceso) en www.someschool.edu en el puerto 80.

2- Servidor HTTP en el equipo: www.someschool.edu esperando una conexión en el puerto 80 acepta la conexión y lo notifica al cliente.

3- Cliente HTTP envía un mensaje de petición (*request*) que contiene una URL a la conexión TCP del socket. El mensaje indica que el cliente quiere objeto:

someDept/home.index

4- Servidor HTTP recibe mensaje de petición, crea el mensaje de respuesta que contiene el objeto pedido y envía el mensaje al socket.

5- El cliente HTTP recibe respuesta que contiene el archivo html. En el archivo html encuentra 10 referencias a objetos jpeg.

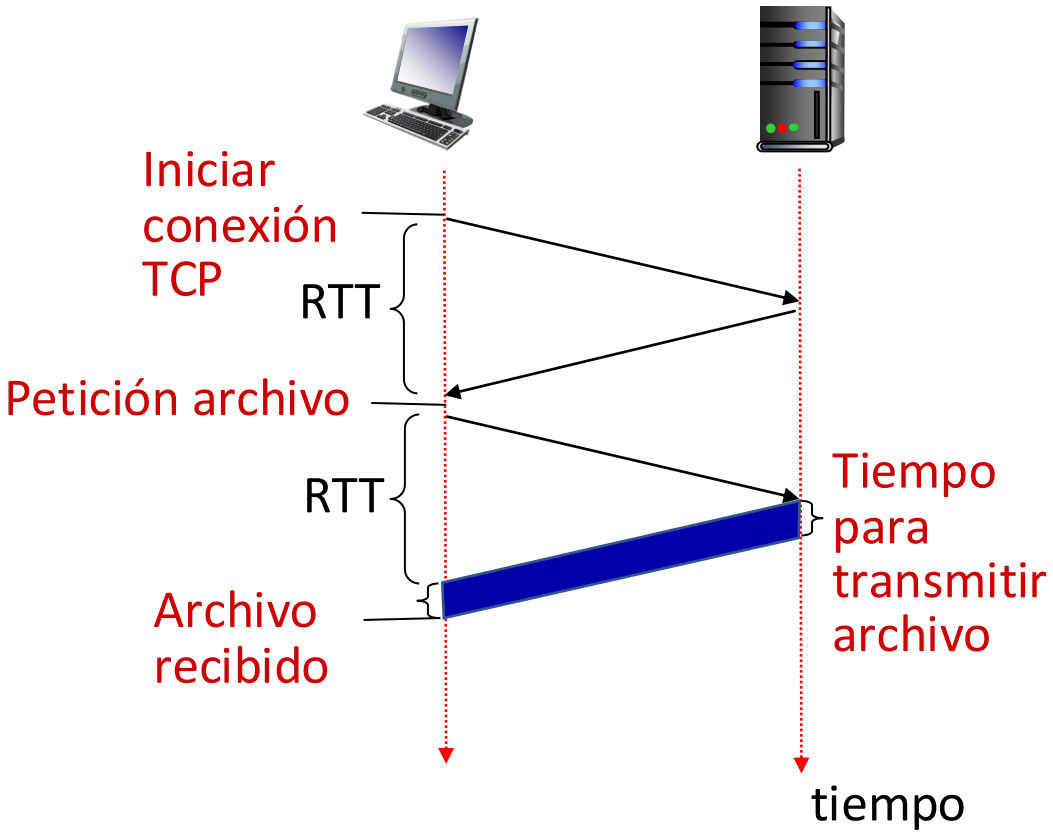
6- Servidor HTTP cierra la conexión TCP.

7- Repetir pasos 1-6 para cada uno de los 10 objetos jpeg.

HTTP

Conexiones no persistentes: tiempo de respuesta

- RTT (*round-trip time*): tiempo que tarda un paquete pequeño del cliente al servidor en ir y volver.
- RTT incluye retrasos de propagación, cola en los conmutadores y routers, retardo de procesado.
- Tiempo de respuesta HTTP:
  - 1 RTT para iniciar conexión TCP.
  - 1 RTT para petición.
  - Tiempo transmisión objeto/archivo.
- Tiempo de respuesta no persistente = 2·RTT + tiempo transmisión archivo.



HTTP

- Inconvenientes **HTTP no persistente**:
  - Requiere 2 RTTs por objeto.
  - Sobrecarga del Sistema Operativo para cada conexión TCP.
  - Navegadores usan múltiples conexiones TCP paralelas para ir a alcanzar objetos referenciados en paralelo.

- **Persistente**:
  - El servidor deja la conexión abierta después de enviar la respuesta.
  - Mensajes que vienen luego enviados sobre la conexión abierta.
  - Cliente envía peticiones tan pronto como encuentra un objeto referenciado.

HTTP: formato del mensaje

- Las RFCs definen el formato del mensaje.
- Dos tipos de mensajes: **mensajes de petición** y **mensajes de respuesta**.

Mensaje típico de petición:

GET /somedir/page.html HTTP/1.1

Host: www.example.org

Connection: close

User-agent: Mozilla/5.0

Accept-language: fr

Mensaje típico de respuesta:

HTTP/1.1 200 OK

Connection: close

Date: Tue, 18 Aug 2015 15:44:04 GMT

Server: Apache/2.2.3 (CentOS)

Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT

Content-Length: 6821

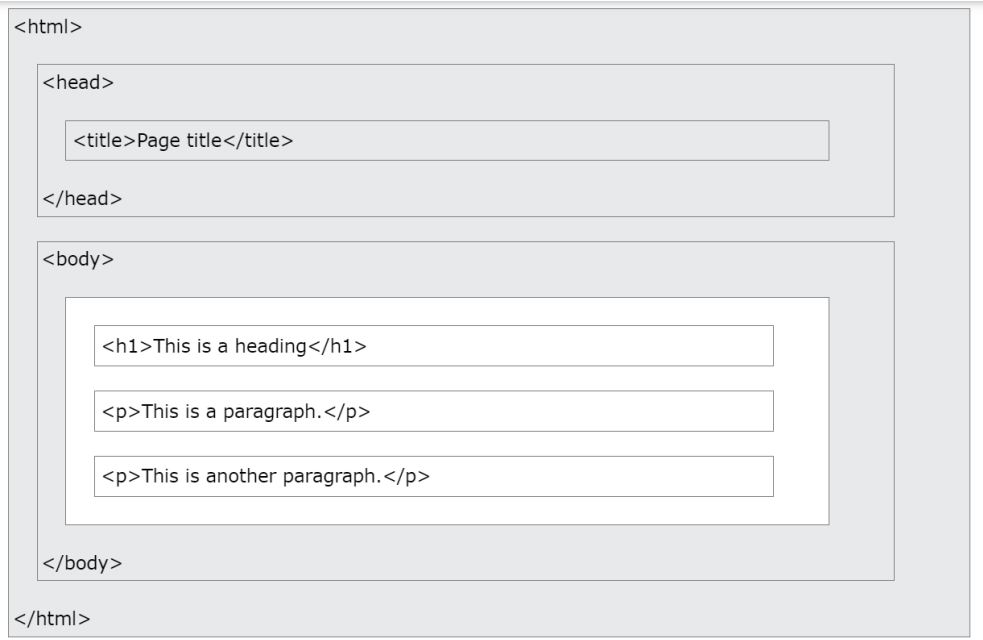
Content-Type: text/html

(data…)

HTML

- HTML es el lenguaje de marcas estándar para crear páginas web.
- HTML describe la estructura de una página web.

Visualización de la estructura de una página web:




```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```

HTML

- Enlaces HTML

```
<a href="https://www.uloyola.es">This is a link</a>
```

- Imágenes HTML

```
<img src= "imagen.jpg" alt= "alternative_text" width="104" height=“142">
```

HTTP mensaje de petición

Carácter de retorno

Carácter de línea

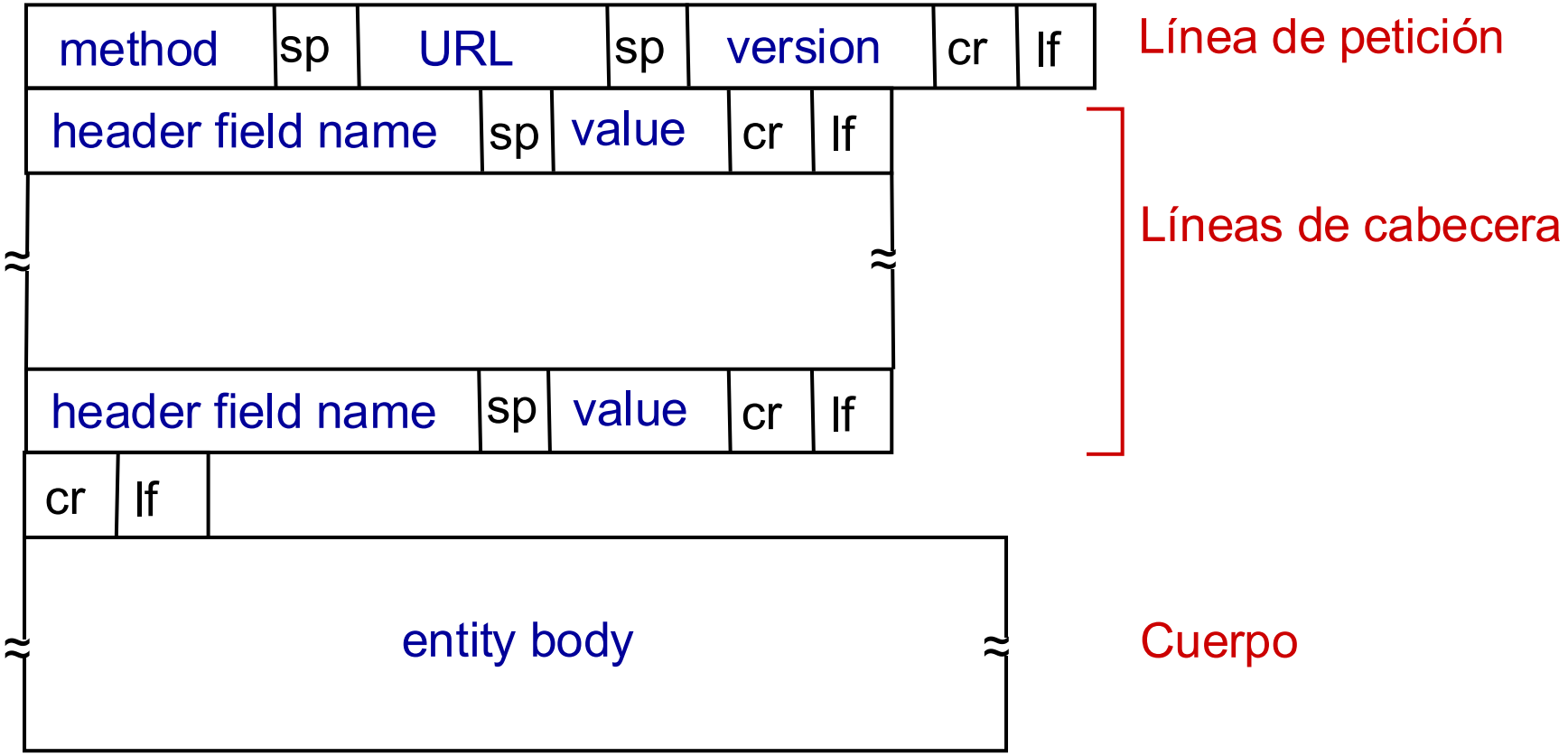
Línea de petición (comandos GET, POST, HEAD)

Líneas de cabecera

Caracteres de retorno y de línea indican el final de las líneas de cabecera

```
GET /index.html HTTP/1.1\r\n
Host: www.google.com\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

HTTP mensaje de petición: formato general



HTTP mensaje de petición

- **GET**

Cuando el navegador pide un objeto. El objeto pedido está identificado en el campo URL (*Uniform Resource Locator*).

- **POST**

Un cliente HTTP lo usa cuando el usuario rellena un formulario, por ejemplo, cuando un usuario proporciona palabras en un motor de búsqueda.

El cuerpo del mensaje contiene lo que el usuario ha introducido en el formulario.

- **HEAD**

Cuando el servidor recibe un mensaje responde con un mensaje HTTP pero omite el objeto pedido.

- **PUT**

Usado con herramientas de publicación web o aplicaciones que necesitan subir objetos a servidores. Permite al usuario subir objetos a una ruta especificada en el servidor web.

- **DELETE**

Permite borrar un objeto en un servidor web.

HTTP mensaje de petición

Una petición generada con un formulario no tiene por qué usar POST obligatoriamente.

Se puede usar GET e incluye los datos en la URL pedida.

Ejemplo: las entradas de dos campos son *monkeys* y *bananas*. La URL será:

www.algunsitio.com/busquedaanimal?monkeys&bananas

URL en detalle

http://www.example.com:80/path/to/myfile.html?key1=value1&key2=value2#SomewhereInTheDocument

HTTP mensaje de respuesta

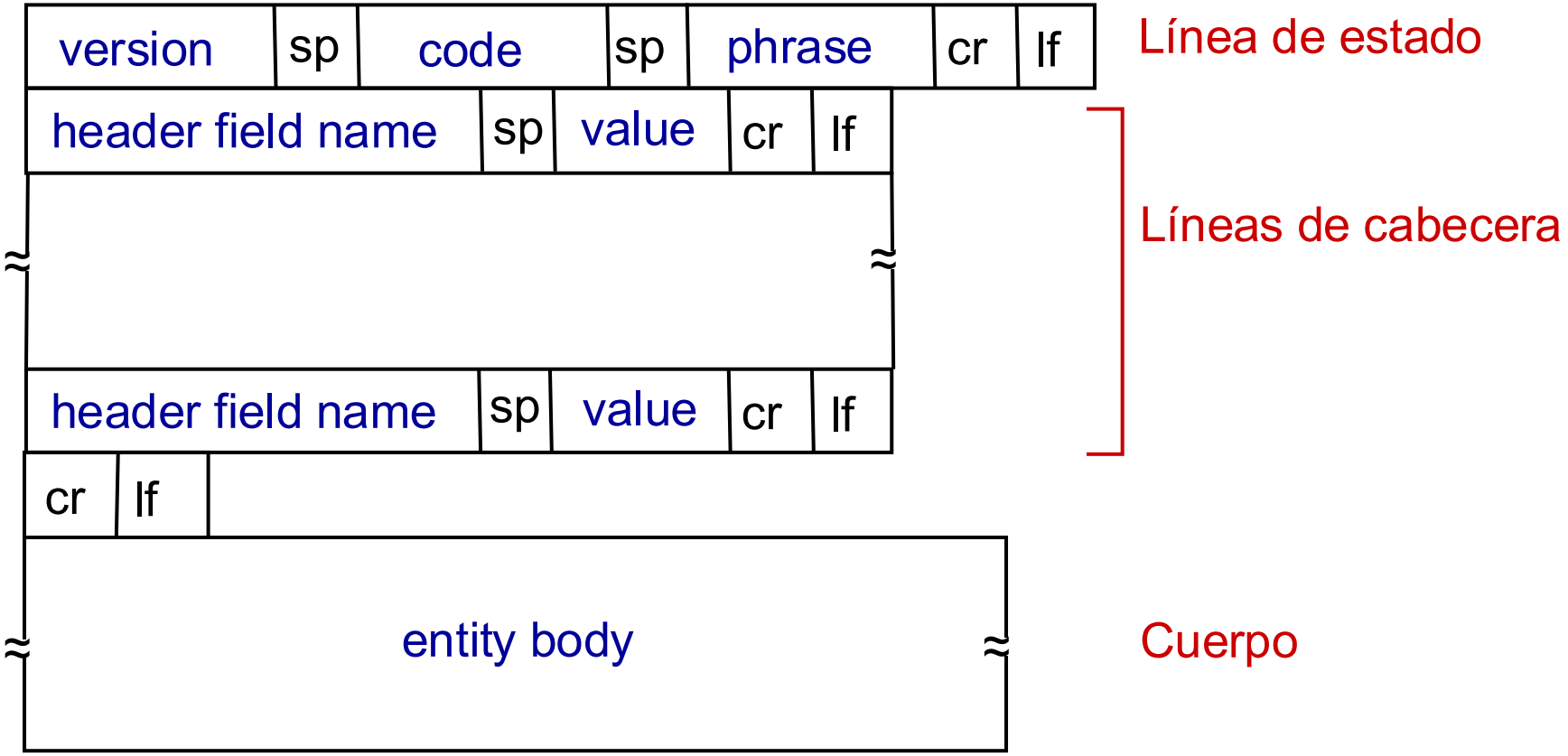
Línea de estado (códigos de estado y frases asociadas)

Líneas de cabecera

Datos, por ejemplo el archivo HTML pedido

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data data data ...
```

HTTP mensaje de respuesta: formato general



HTTP mensaje de respuesta

- **200 OK**

La petición ha tenido éxito y la información es devuelta en la respuesta.

- **301 Moved Permanently**

El objeto pedido ha sido movido permanentemente. La nueva URL es especificada en la cabecera *Location* del mensaje de respuesta. El cliente automáticamente recupera la nueva URL.

- **400 Bad Request**

Código de error genérico indicando que la petición no puede ser entendida por el servidor.

- **404 Not Found**

El documento pedido no existe en el servidor.

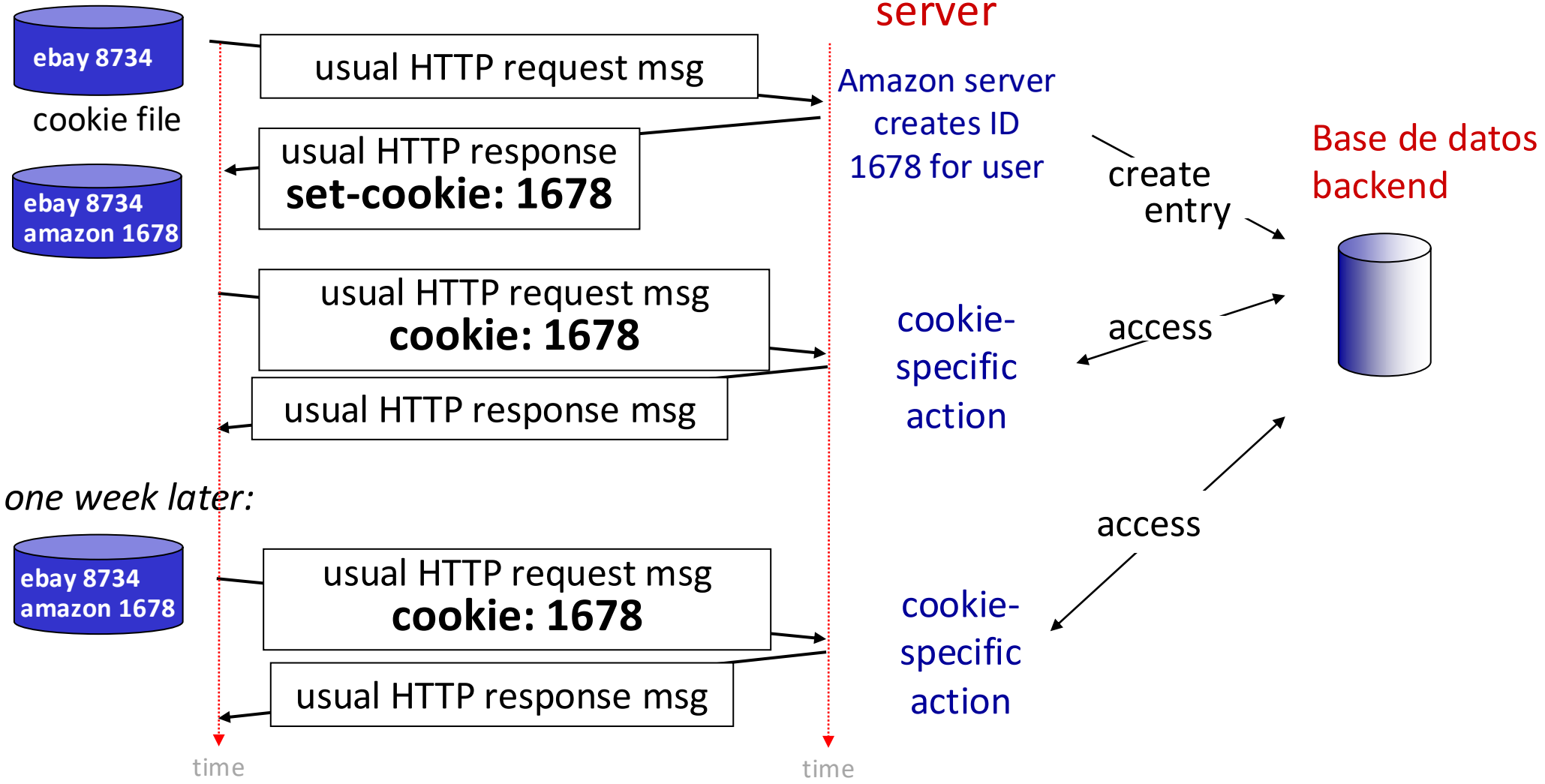
- **505 HTTP Version Not Supported**

La versión del protocolo HTTP no es soportada por el servidor.

Interacción usuario-servidor: *Cookies*

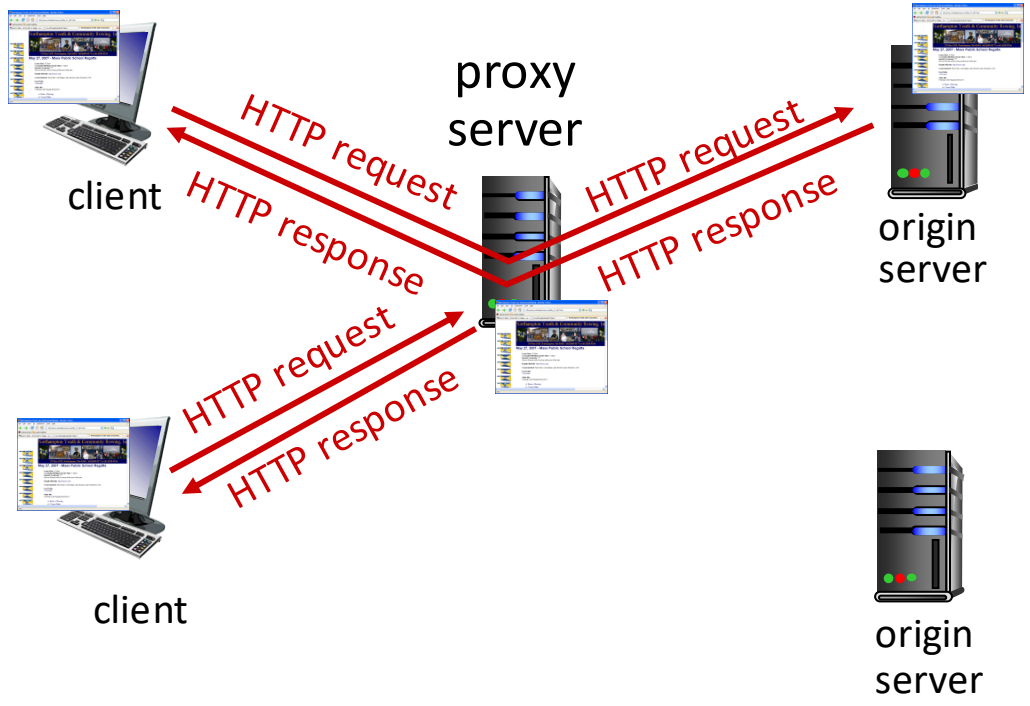
- A veces es conveniente para un sitio web identificar a los usuarios, ya que el servidor desea restringir el acceso a los usuarios o porque quiere servir contenido en función de la identidad. Por eso, HTTP usa *cookies*.
- *Cookies* (RFC 6265) permite a los sitios web mantener un registro de los usuarios.
- Las *cookies* tienen 4 componentes:
  - Una línea de cabecera *cookie* en el mensaje de respuesta HTTP.
  - Una línea de cabecera *cookie* en el mensaje de petición HTTP.
  - Un archivo *cookie* en el sistema final del usuario y manejado por el navegador.
  - Base de datos *backend* en el sitio web.

Interacción usuario-servidor: *Cookies*



Web caching

- Una web cache (Servidor proxy) satisface las peticiones HTTP en nombre del servidor web origen.
- Tiene su propio disco de almacenamiento y mantiene copia de los objetos pedidos.
- Un navegador puede ser configurado de forma que todas las peticiones HTTP son dirigidas primero al servidor proxy.



- Cliente y servidor al mismo tiempo.
- Instalado por el ISP.
- Reduce el tiempo de respuesta para peticiones de clientes.
- Reduce el tráfico web.

Sitios web estáticos y dinámicos

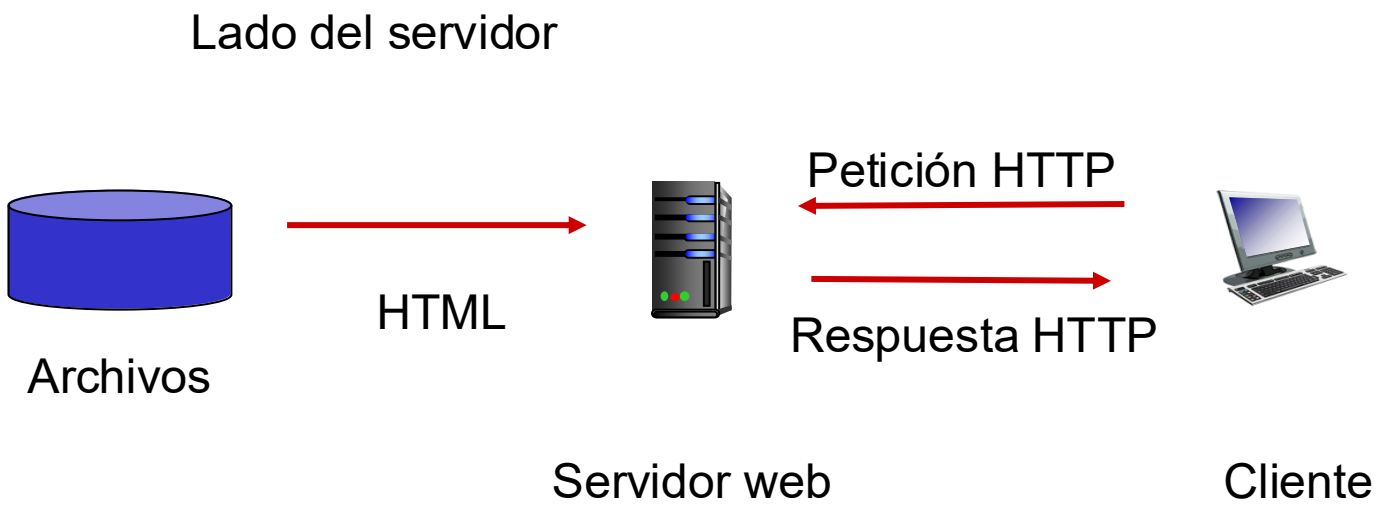
Servidor web puede referirse a:

Hardware: máquina que corre un software y almacena los archivos que componen el sitio web (documentos HTML, CSS, JS, …)

Software: programa que controla cómo los usuarios acceden a los archivos. Este servidor interpreta el protocolo HTTP y las URL.

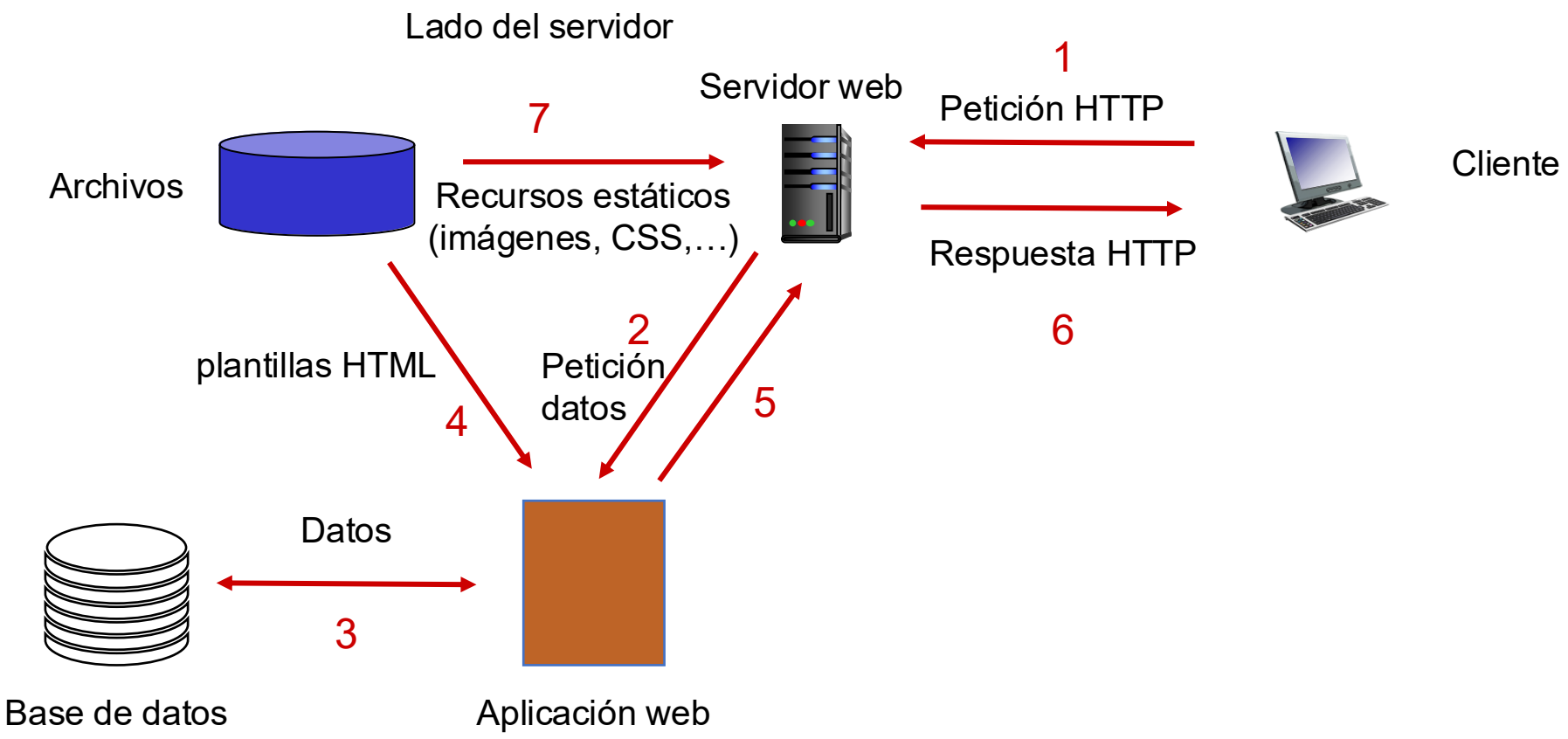
Sitios web estáticos y dinámicos

Sitio web estático: Contenido fijo. No se requiere de base de datos ni de programación.



Sitios web estáticos y dinámicos

Sitios web dinámicos: el contenido es fácilmente modificable. Se construye usando programación.



Frontend y *backend*

- ***Frontend***: es la parte web que podemos ver y con la que podemos interactuar.
  - Desarrollo web.
  - Diseño web.
- El *frontend* es una combinación de:
  - **HTML** (estructura).
  - **CSS** (estilos).
  - **JavaScript** (añade interactividad).

Frontend y *backend*

- ***Backend***: se refiere al lado de una aplicación que está en el servidor, y a todo lo que se comunica con el navegador. Consta de al menos tres elementos:
  - Servidor web (procesa las peticiones HTTP).
  - Servidor de aplicaciones (gestiona la aplicación).
  - Base de datos.
- Lenguajes de programación del *backend*:
  - **PHP:** lenguaje de *scripting* que permite generar contenido dinámico. Puede ser incrustado en HTML.
  - **Java**
  - **JavaScript**
  - **Python**
  - **Ruby**
  - **.NET**

Contenido

1- La web y HTTP

2- DNS

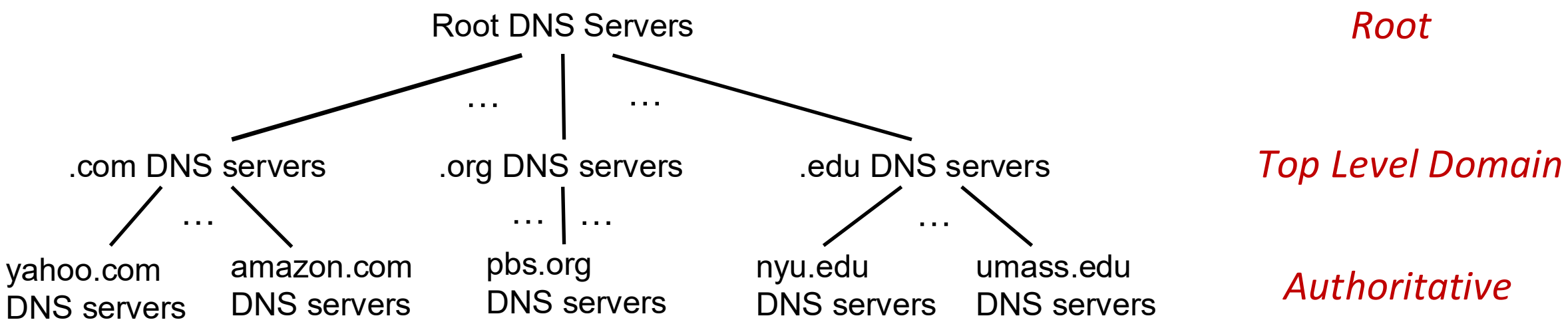
3- *Streaming* de video y redes de distribución de contenido

4- Programación de sockets: creando aplicaciones de red

DNS: Domain Name System

- Hay dos formas de identificar un equipo, por el *hostname* o por la dirección IP.
- El principal objetivo de DNS (*Domain Name System*) es la de traducir el *hostname* a una dirección IP.
- DNS es una base de datos distribuida implementada en una jerarquía de servidores DNS.
- DNS es un protocolo de la capa de aplicación.
- Funciona sobre UDP (puerto 53).

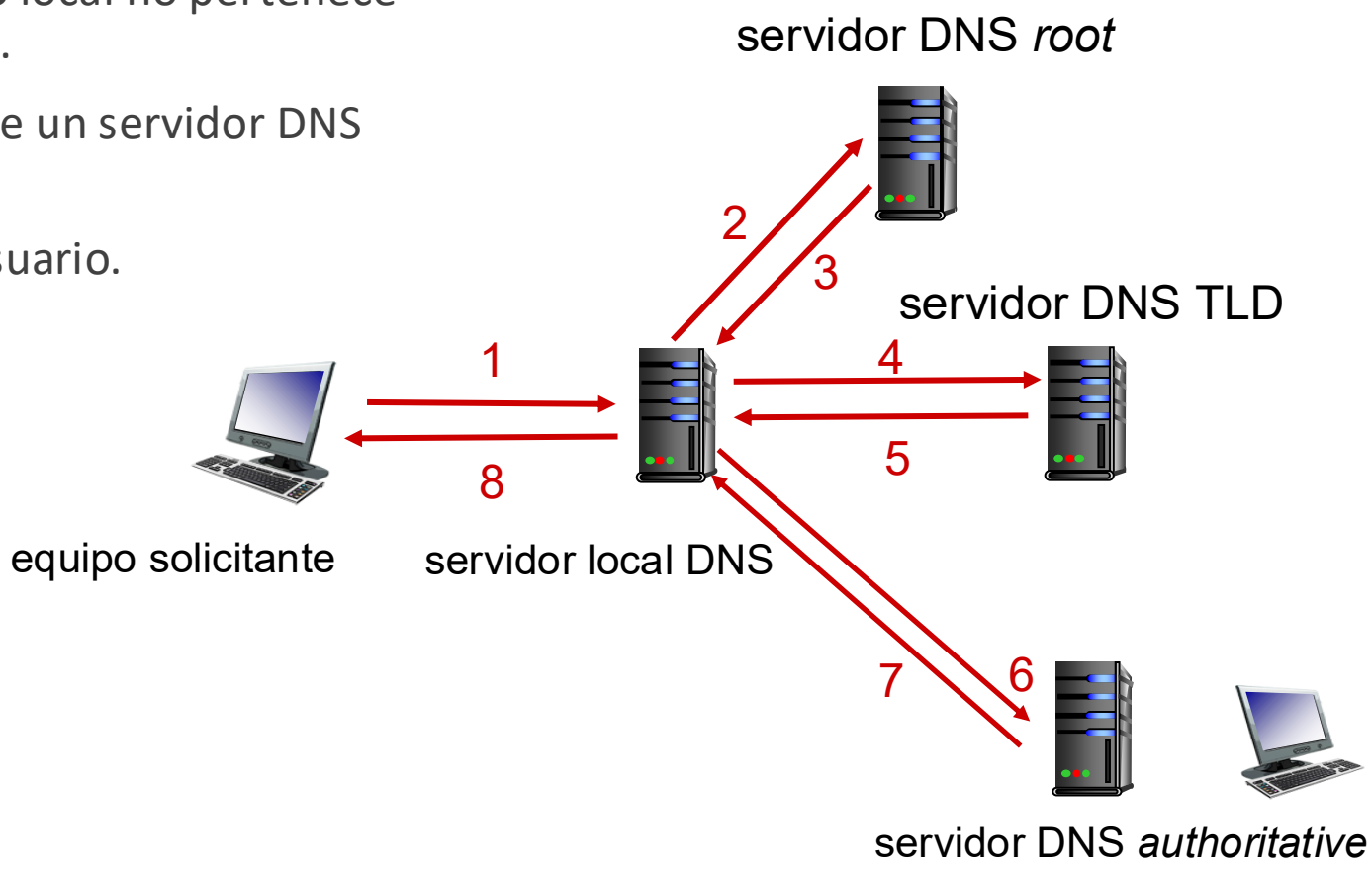
DNS, una base de datos jerárquica y distribuida



- Más de 1000 servidores *root* repartidos por todo el mundo.
- Servidores TLD: por cada dominio. com, org, edu, net, gov,… y por países es, fr, ca, jp,..
- Servidores DNS *Authoritative*: Cada organización con equipos accesibles públicamente (como servidores web o servidores de correo) deben proporcionar un registro DNS accesible con la correspondencia entre nombre de equipos y direcciones IP.

DNS, petición de IP

- **Servidor DNS** local no pertenece a la jerarquía.
- Cada ISP tiene un servidor DNS local.
- Cercano al usuario.



DNS *caching*: cuando un servidor DNS recibe una respuesta DNS puede guardarla en su memoria local

Contenido

1- La web y HTTP

2- DNS

3- *Streaming* de video y redes de distribución de contenido

4- Programación de sockets: creando aplicaciones de red

Video *streaming* y redes de distribución de contenidos

- Tráfico video *streaming*: mayor consumidor del ancho de banda de Internet. Ejemplos: Netflix, YouTube, Amazon Prime, TikTok,…
- Reto: ¿Cómo alcanzar a tantos usuarios alrededor del mundo?
- Un único mega servidor de video no funcionaría. ¿Por qué?
- Otro reto es la heterogeneidad. Diferentes usuarios tienen diferentes capacidades de ancho de banda, red cableada o red móvil, …

Video *streaming* y redes de distribución de contenidos

- Tráfico video *streaming*: mayor consumidor del ancho de banda de Internet. Ejemplos: Netflix, YouTube, Amazon Prime, TikTok,…
- Reto: ¿Cómo alcanzar a tantos usuarios alrededor del mundo?
- Un único mega servidor de video no funcionaría. ¿Por qué?
- Otro reto es la heterogeneidad. Diferentes usuarios tienen diferentes capacidades de ancho de banda, red cableada o red móvil, …
- Solución: **Infraestructura nivel de aplicación distribuida.**

Multimedia: video

- **Video**: secuencia de imágenes mostradas a una tasa constante, por ejemplo 24 imágenes/s.
- **Imagen digital:** tabla de píxeles. Cada pixel es representado por bits.
- Codificación: usar redundancia dentro y entre imágenes para reducir el número de bits usados para codificar la imagen.
  - Espacial: dentro de la imagen.
  - Temporal: entre imágenes (de una imagen a otra).

Ejemplo codificación espacial: En vez de enviar N valores del mismo color (todo violeta), enviamos solo dos valores: valor del color (*violeta)* y número de valores repetidos *(*N).

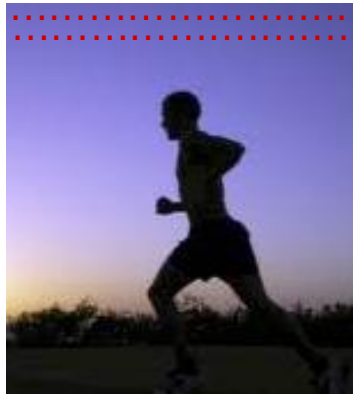


imagen *i*

imagen *i+1*

Ejemplo codificación temporal: en vez de enviar la imagen completa en i+1, enviamos solo las diferencias con i.

Multimedia: video

- **Tasa de codificación fija**.
- **Tasa de codificación variable**. Cambia según la codificación espacial y temporal cambia.
- Ejemplos:
  - Video en Internet comprimido típicamente varía entre 100 kbps para baja calidad hasta 4 Mbps para películas de alta definición.
  - Compresión para tener diferentes versiones del mismo video, con diferentes calidades, ej: 300 kbps, 1 Mbps y 3 Mbps.

Ejemplo codificación espacial: En vez de enviar N valores del mismo color (todo violeta), enviamos solo dos valores: valor del color (*violeta)* y número de valores repetidos *(*N).

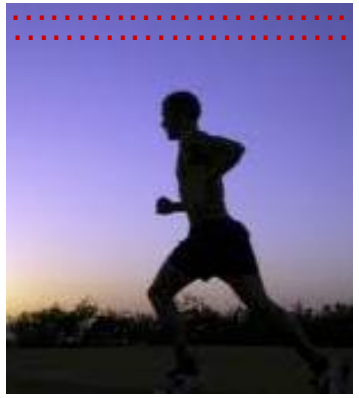


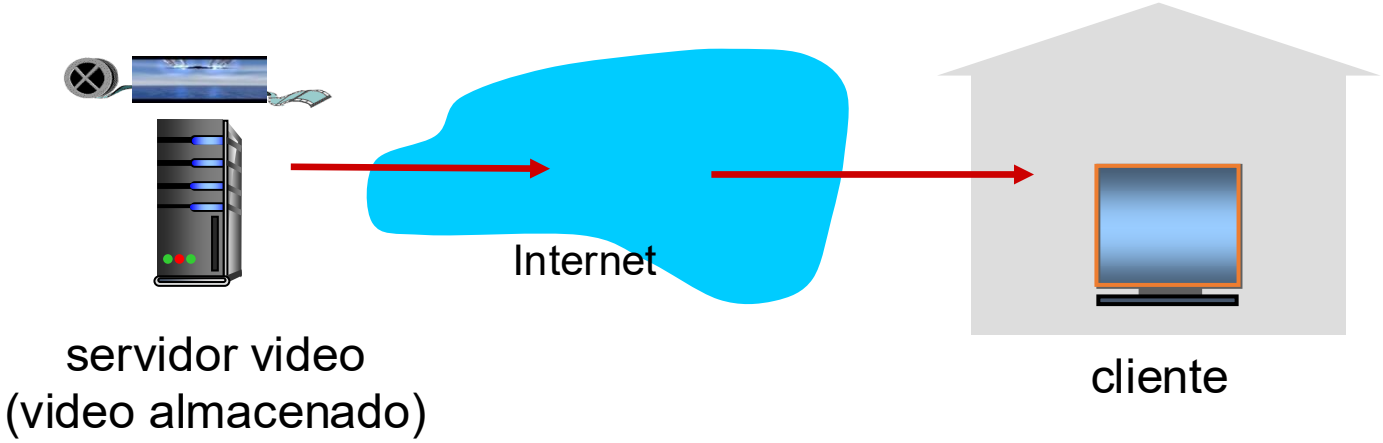
imagen *i*

imagen *i+1*

Ejemplo codificación temporal: en vez de enviar la imagen completa en i+1, enviamos solo las diferencias con i.

Multimedia: video

- Reto 1: El ancho de banda entre cliente y servidor puede variar en función del tiempo dependiendo de la congestión del tráfico de red (en casa, red acceso, entre *routers*, en el servidor de video,…).
- Reto 2: pérdida de paquetes debido a la congestión retrasará la visualización, resultando en un video de baja calidad.



Streaming multimedia: DASH

DASH: *Dynamic Adaptative Streaming over HTTP*.

- Servidor:
  - divide el archivo de video en múltiples trozos.
  - Cada trozo es almacenado y codificado a diferentes tasas.
  - El servidor contiene lo que se llama ***manifest file***: proporciona URL correspondientes a los diferentes trozos.
- Cliente:
  - Mide periódicamente el ancho de banda entre cliente y servidor.
  - Consultando *manifest*, pide un trazo en cada instante de tiempo, eligiendo la tasa de codificación máxima soportada por el ancho de banda dado.
  - Puede elegir diferentes tasas de codificación en diferentes puntos de tiempo dependiendo del ancho de banda disponible.

Streaming multimedia: DASH

DASH: *Dynamic Adaptative Streaming over HTTP*.

- Inteligencia en el cliente. El cliente determina:
  - Cuando pedir un trozo.
  - Cuál tasa de codificación pedir.
  - Dónde pedir el trozo (puede pedir de una URL correspondiente a un servidor más cercano al cliente o con mayor ancho de banda disponible).

***Streaming* video = codificación + DASH + almacenamiento.**

Redes de distribución de contenido

CDNs (Content Distribution Networks)

- Reto: cómo enviar y recibir contenido (millones de videos y cientos de miles de usuarios simultáneos).
- **Opción 1**: Un único mega servidor
  - Único punto de fallo.
  - Punto de congestión de red.
  - Larga distancia a clientes.
  - Múltiples copias de video enviadas por el mismo enlace.

Redes de distribución de contenido

CDNs (Content Distribution Networks)

- Reto: cómo enviar y recibir contenido (millones de videos y cientos de miles de usuarios simultáneos).
- **Opción 1**: Un único mega servidor
  - Único punto de fallo.
  - Punto de congestión de red.
  - Larga distancia a clientes.
  - Múltiples copias de video enviadas por el mismo enlace.

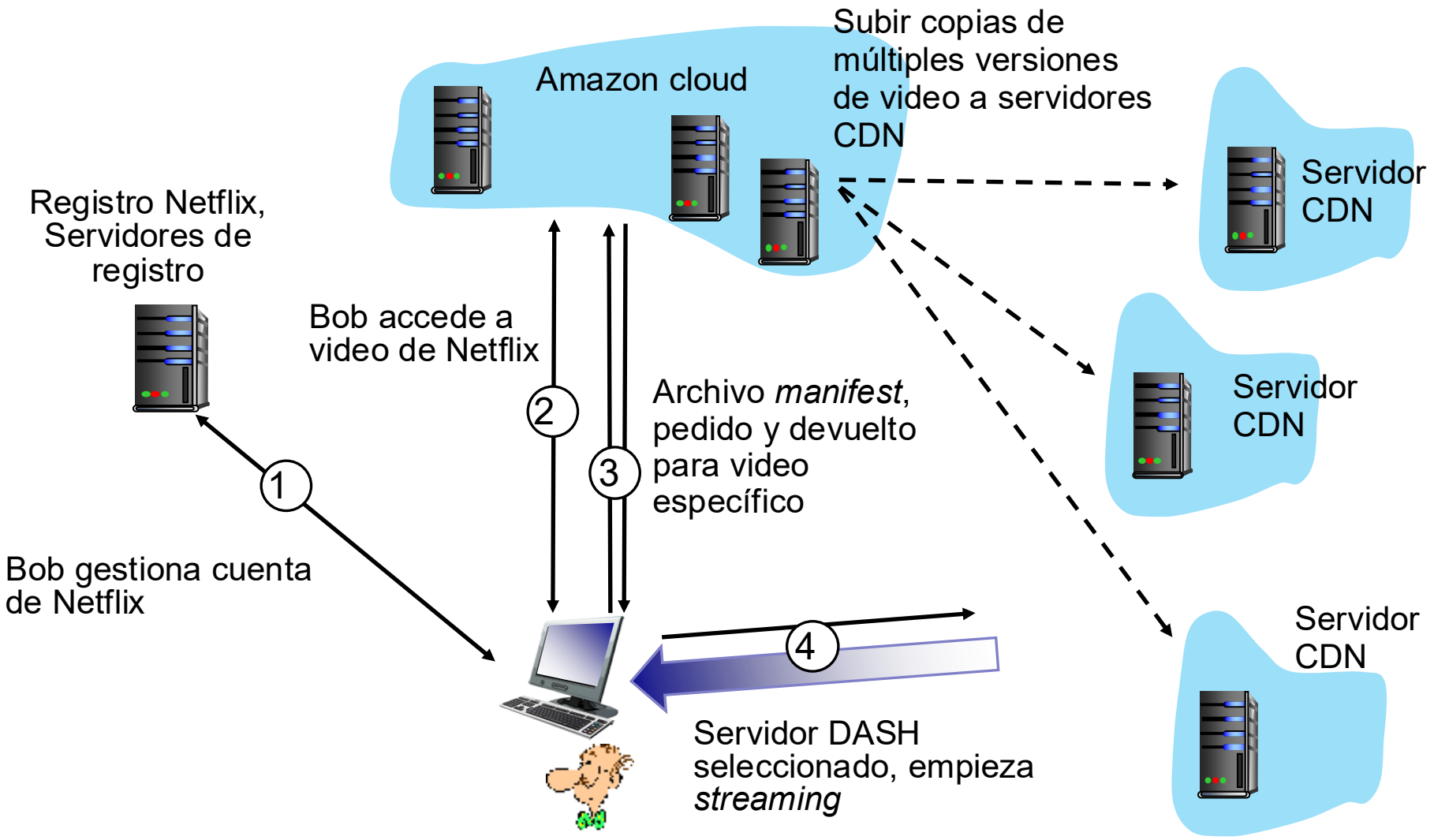
Esta solución no es viable

Redes de distribución de contenido

CDNs (Content Distribution Networks)

- Reto: cómo enviar y recibir contenido (millones de videos y cientos de miles de usuarios simultáneos).
- **Opción 2**: almacenar y servir múltiples copias de video en múltiples sitios distribuidos geográficamente (CDNs).

Redes de distribución de contenido: Netflix



Contenido

1- La web y HTTP

2- DNS

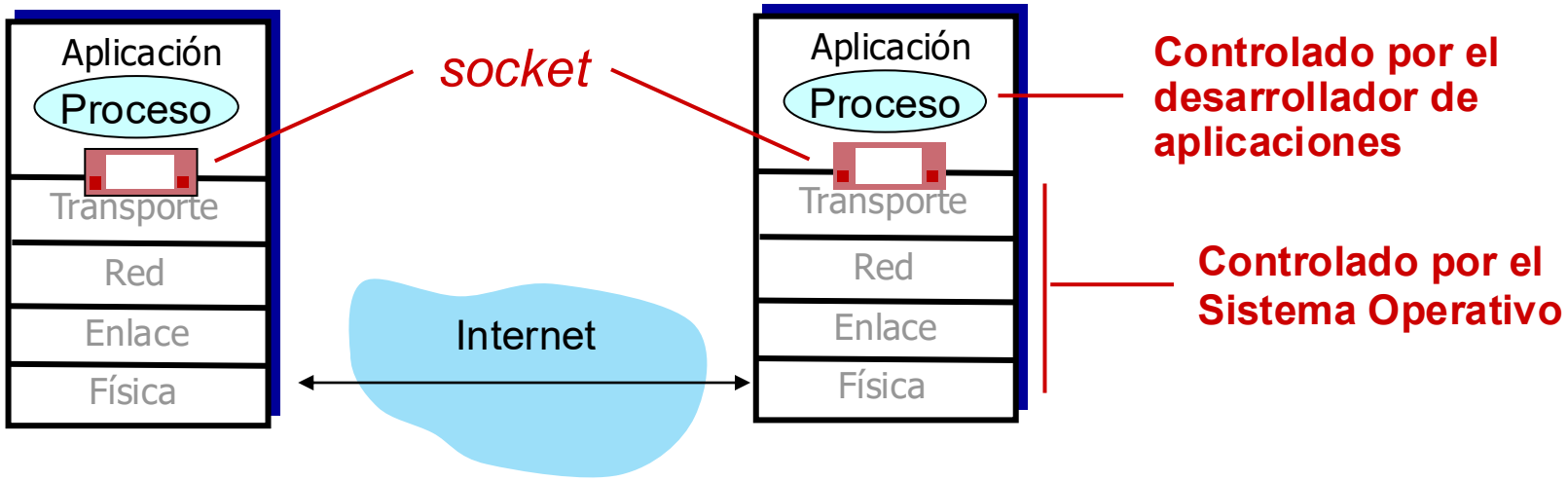
3- *Streaming* de video y redes de distribución de contenido

4- Programación de sockets: creando aplicaciones de red

Programación de sockets

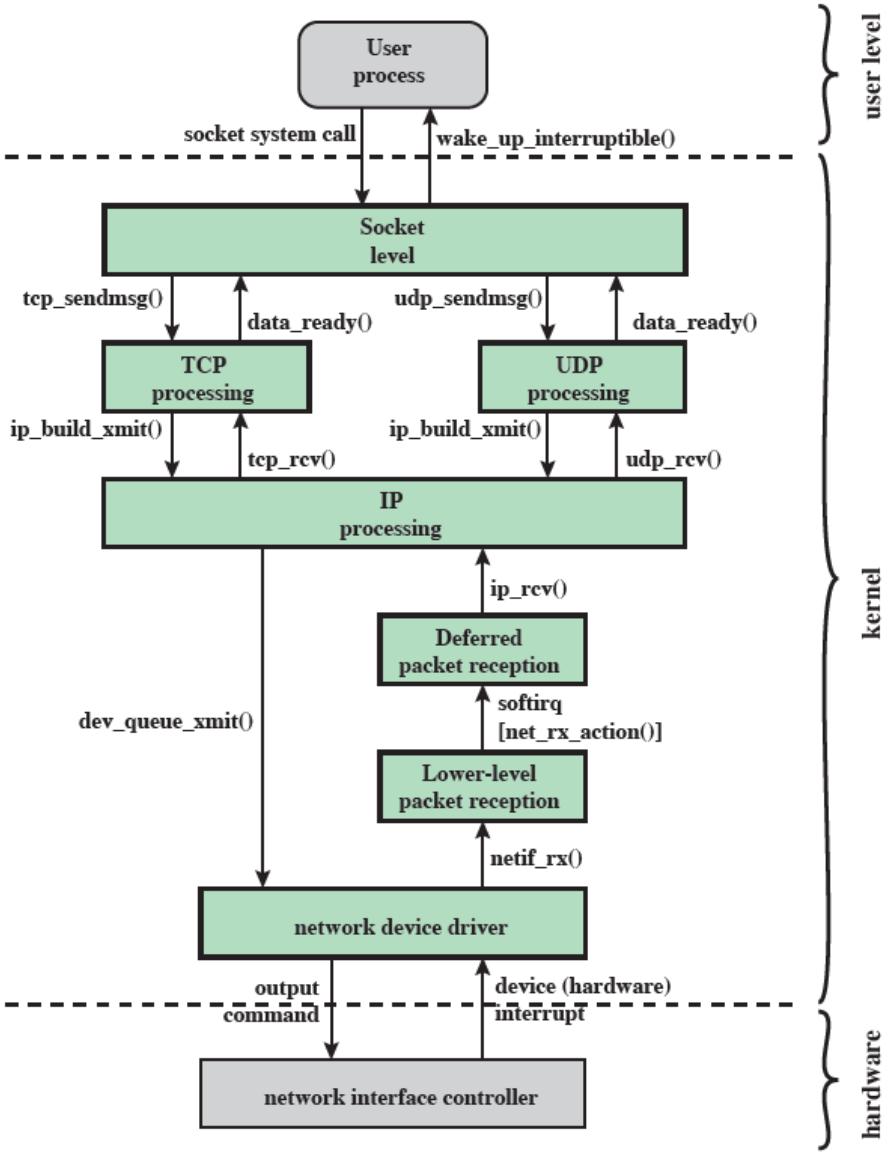
Objetivo: aprender cómo construir aplicaciones cliente y servidor que se comunican usando sockets (en **lenguaje C**).

Socket: puerta entre procesos de aplicación y protocolos de transporte (punto a punto).



Programación de sockets

Componentes del kernel de Linux para el procesado TCP/IP



Programación de sockets

Encapsulación de datos:

- Los datos se encapsulan en una cabecera por la aplicación (Ejemplo: FTP).
- Los datos y cabecera se encapsulan por la capa siguiente (UDP).
- Luego ocurre igual con el protocolo IP. Prepara el paquete para la ruta.
- Finalmente, Ethernet u otro estándar de la capa de enlace/física.

Programación de sockets

Los servicios de transporte:

- Proporcionan una **comunicación lógica** entre **procesos** que corren en diferentes máquinas.
- Diferencias entre la capa de transporte y la capa de red:
  - **Capa de transporte**: comunicación entre **procesos**.
  - **Capa de red**: comunicación entre ***hosts***.

Programación de sockets

Dos tipos de sockets para dos servicios de transporte:

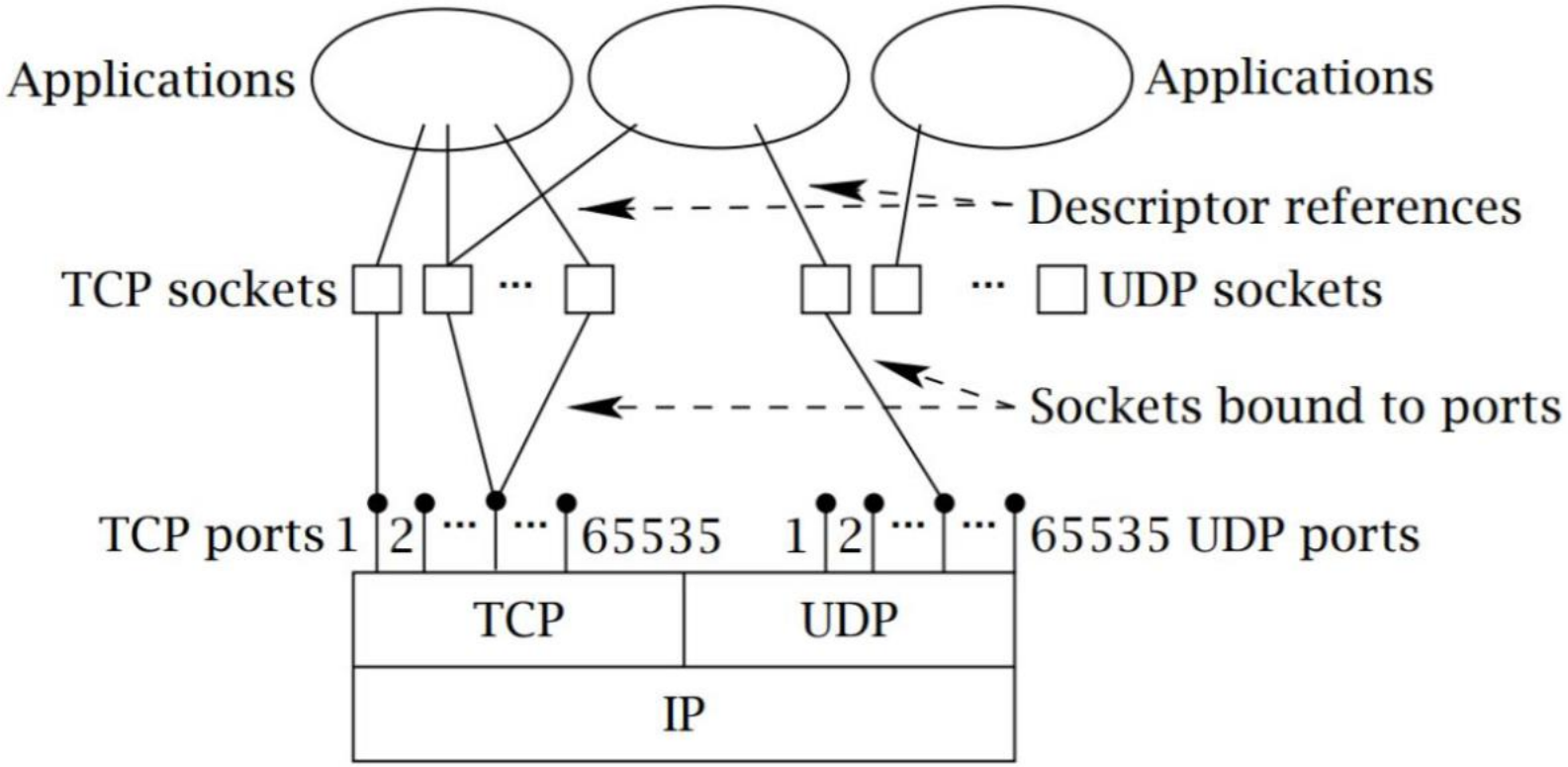
UDP (*User Datagram Protocol*)

- Proporciona una transferencia **no fiable** de grupos de bytes (datagramas) entre cliente y servidor.
- No recupera errores en la transmisión.
- Sólo extiende el servicio ofrecido por la capa IP a los programas (procesos). IP solo envía información de un host a otro.
- Los datos transmitidos pueden ser perdidos o entregados desordenados.

TCP (*Transmission Control Protocol*)

- **Fiable**, orientada a un flujo de bytes.
- Detecta y recupera errores, ordenación y pérdida de paquetes.
- Proporciona un **canal seguro** de envío de bytes, sin que la capa de aplicación tenga que preocuparse de estos problemas.

Programación de sockets



Programación de sockets

Consideraciones importantes:

- Un programa puede tener varios sockets en uso al mismo tiempo.
- Varios programas pueden estar usando el mismo socket al mismo tiempo.
- Cada socket tiene asociado un puerto local, TCP o UDP que es usado para redirigir los paquetes entrantes a la aplicación pertinente.
- Más de un socket puede estar asociado a un puerto local.

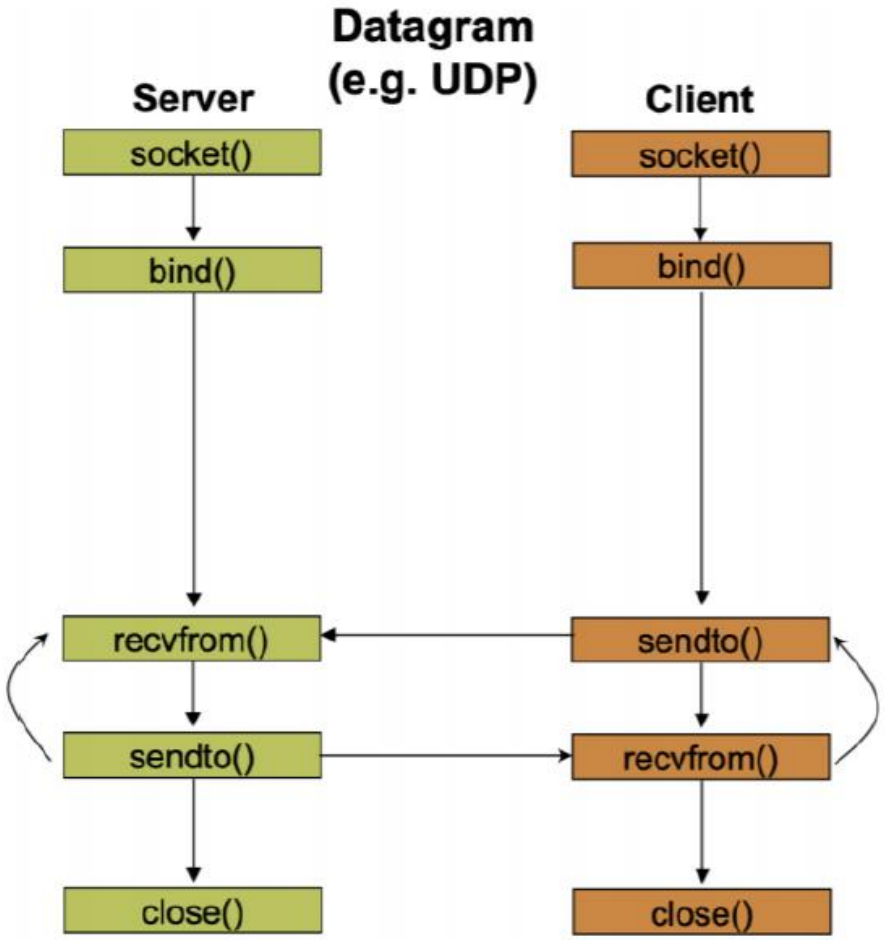
Socket UDP

socket(): crea e inicializa el socket

bind(): asocia un socket con una dirección IP local y un número de puerto.

sendto() and **recvfrom()** son usados para enviar y recibir datos.

close(): cierra el socket.



Socket TCP

socket(): crea e inicializa el socket

bind(): asocia un socket con una dirección IP local y un número de puerto.

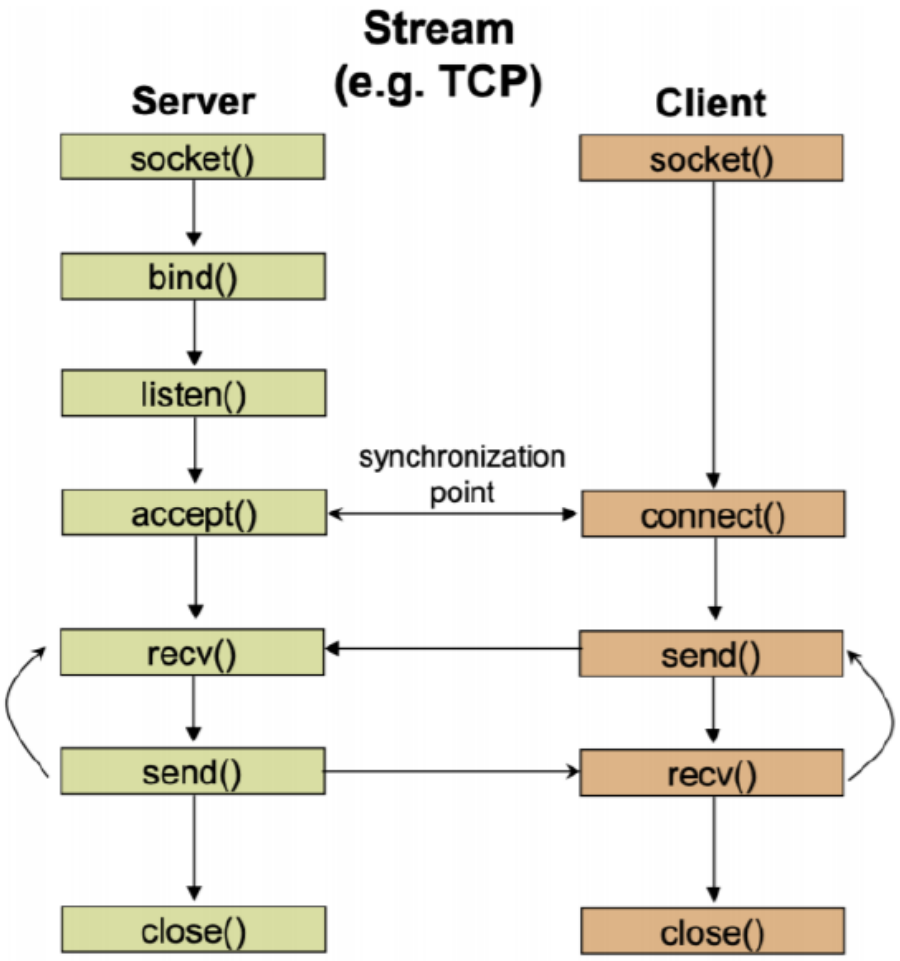
listen(): es usado en el servidor para que un socket TCP escuche nuevas conexiones.

accept(): acepta una conexión desde el cliente.

connect(): usado en el cliente para establecer la dirección remota y el puerto.

send() and **recv()** son usados para enviar y recibir datos.

close(): cierra el socket.



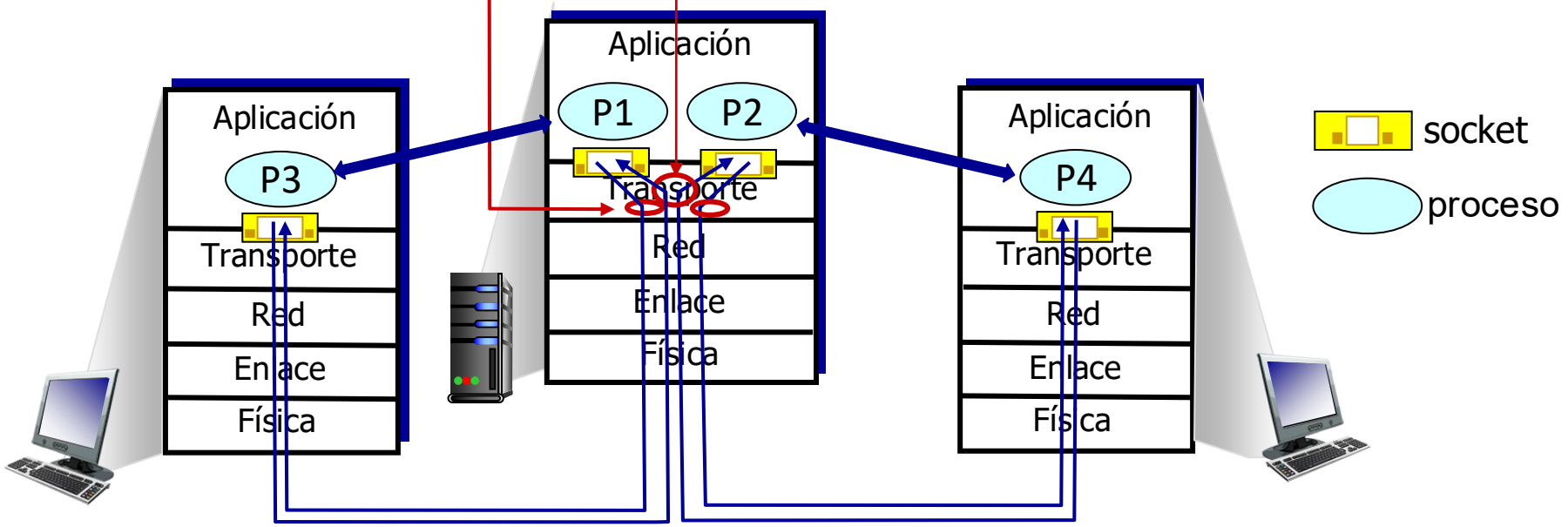
Multiplexado/demultiplexado

Multiplexado como remitente. Maneja datos desde múltiples sockets.

Añade cabecera de transporte.

Usada después para demultiplexado.

Multiplexado como receptor. Maneja la información de cabecera para entregar los segmentos recibidos al socket correcto.



Demultiplexado no orientado a conexión

- Cuando el *host* receptor recibe un segmento UDP:
  - Comprueba número de puerto de destino en segmento.
  - Direcciona el segmento UDP al socket con ese número de puerto.
- Datagramas IP/UDP con el **mismo número de puerto destino,** pero **diferente dirección IP remitente** y/o **número de puerto remitente** serán dirigidos al **mismo socket** en el *host* que recibe.
- **UDP**: demultiplexado usando solo número de puerto destino.

Demultiplexado orientado a conexión

- El socket **TCP** está identificado por:
  - Dirección IP fuente.
  - Número de puerto fuente.
  - Dirección IP destino.
  - Número de puerto destino.
- Demultiplexado: el receptor **usa los cuatro valores** para direccionar el segmento al socket apropiado.
- Cada servidor puedo soportar varios sockets TCP simultáneos:
  - Cada socket identificado por sus cuatro valores.
  - Cada socket asociado con un cliente diferente.

Universidad
LOYOLA

Universidad
LOYOLA